

Topic: Polynomial Representation and Addition -- Self-Learning

Q. Explain how a Singly Linked List can be used to represent a polynomial. Describe the algorithm to add two polynomials represented using linked lists with a suitable example.

[DEF] Definition to Remember

A polynomial (e.g., $3x^4 + 2x^2 + 5$) can be efficiently represented using a Singly Linked List where each node stores a non-zero term's coefficient and exponent. This is vastly superior to arrays for sparse polynomials, as it avoids wasting memory on zero-coefficient terms.

Polynomial Representation & Addition (Self-Learning)

Each node in the linked list represents one term and is divided into three parts:

1. Co

Structure in C:

```
struct
{
    in
    in
    st
};
```

(Nodes are kept sorted in descending order of their exponents to make addition easier).

Example Diagram for $5x^3 + 4x^1 + 2x^0$:

Head

2. Polynomial Addition Algorithm

To add Poly1 and Poly2, traverse both lists simultaneously using pointers p1 and p2. Compare their exponents:

- * Case 1: $p1 \rightarrow \text{exponent} > p2 \rightarrow \text{exponent}$

Appen

After the loop, append any remaining terms from either p1 or p2 to the Result list.

3. Example Trace

- * Poly1: $3x^2 + 5x + 2$

* Po

4. Advantages

- * Memory Efficiency: Only non-zero terms are stored. (An array for $x^{100} + 1$ requires 101 slots; a linked list requires only 2 nodes).

* Dy

[MUST-WRITE] Must-Write Points (for 10 marks)

1. Arrays waste memory for sparse polynomials (many zero terms). Linked Lists only store non-zero terms.
2. Node structure: int coefficient, int exponent, and struct PolyNode *next.
3. Terms are kept sorted in descending order of exponents.
4. Addition Logic ($p1 > p2$): Add $p1$ to result, advance $p1$.
5. Addition Logic ($p1 < p2$): Add $p2$ to result, advance $p2$.
6. Addition Logic ($p1 == p2$): Add coefficients together. If sum $!= 0$, add to result. Advance both $p1$ and $p2$.
7. Append any leftover terms at the end of the lists.

[RECALL] Quick Recall

Poly LL -> Node(coeff, exp, next) -> Sorted by Exp (descending) -> Add: if $exp1 > exp2$ (copy 1), if $exp1 < exp2$ (copy 2), if $exp1 == exp2$ (add coeffs) -> Saves memory for sparse polys